

Crop Advisory v4 — Final Demo Guide

2026-09-23 · @Someone · Setup, settings, start-up, viewing data and troubleshooting in one place

What's new in this edition

Server uptime — the dashboard top bar now shows how long the backend process has been running, as a duration and a wall-clock start time (§10).

Timestamped logs — every backend log line now carries a YYYY-MM-DD HH:MM:SS wall-clock timestamp, and every reading and SMS attempt is logged as it happens, not just failures (§10, §17).

Activity log — a collapsible section on each device's dashboard page listing its last 10 readings and last 10 SMS attempts (§10).

Test SMS with real data — a new button that force-sends a real SMS built from a device's actual latest reading, bypassing the cooldown — for checking wording and delivery against genuine field numbers (§12, §14). The SMS-text disclosure under the SMS banner now starts collapsed, expanding only on click.

Your current working setup (verified 2026-09-23)

Network: phone hotspot Cookie (2.4 GHz). PC Wi-Fi address 10.144.56.125, ESP32 address 10.144.56.21.

ESP32 config.h: `BACKEND_URL "http://10.144.56.125:8000/sensor-data", DEVICE_ID "FIELD-001", DEVICE_KEY "demo-key-123".`

Backend .env: `SMS_PROVIDER=textbee, EXPECTED_DEVICE_KEY=demo-key-123, SOWING_DATE=2026-06-15, ALERT_COOLDOWN_MINUTES=5.`

Dashboard on the PC: `http://127.0.0.1:8000/dashboard`. On the phone:

`http://10.144.56.125:8000/dashboard`.

The PC address can change whenever the hotspot restarts. Always re-check with `ipconfig` before a demo (§1, §5).

Contents

1. Quick Start — Every Time You Run the System	3
2. Overview & Architecture	3
3. Hardware & Wiring	4
4. Flashing the ESP32	5
4.1 Install board support	5
4.2 Install libraries	5
4.3 Calibrate all three analog sensors	5
4.4 Create config.h	5
4.5 Upload the main firmware	6
5. Network Setup — Getting the ESP32 to Reach the PC	7
5.1 Reading ipconfig	7
5.2 Which address to use	7
5.3 Open the firewall (once, as Administrator)	7
5.4 Five-second network test	8
6. Backend Setup on Windows (VS Code, first time only)	8
7. Changing Settings — Where Everything Lives	8
7.1 Common changes, step by step	9
8. Environment Variables Reference (.env)	11
9. SMS Language	12
9.1 How the language is chosen	12
9.2 Changed .env to Hindi but still getting English?	12
10. Viewing the Data	13
What the dashboard shows	13
11. Codebase Tour	13
12. API Endpoint Reference	15
13. Alert Codes Reference	15
14. Running the Live Demo — Real Hardware	17
15. Running the Live Demo — No Hardware	17
16. Serial Monitor Messages Decoded	18
17. Troubleshooting	19
18. Known Limitations & Roadmap	21

1. Quick Start — Every Time You Run the System

Follow these in order. Each step says what success looks like, so you know immediately if something is off. Details for every step are in the later sections.

#	Do this	Success looks like
1	Phone: turn on the hotspot Cookie. Keep it on 2.4 GHz (on Android, turn on <i>Maximise compatibility</i> / 2.4 GHz band). Make sure the textbee app is running and the SIM has SMS balance.	Hotspot shows as on; textbee app shows the device as connected.
2	PC: connect Wi-Fi to Cookie. Open Command Prompt, run <code>ipconfig</code> , read the IPv4 Address under Wireless LAN adapter Wi-Fi.	Address is 10.144.56.125. If it's different, update <code>BACKEND_URL</code> in <code>config.h</code> and re-upload the sketch (§4.5).
3	VS Code: open the <code>crop-advisory-backend</code> folder itself. In the terminal run <code>start_server.bat</code> .	Terminal shows Uvicorn running on <code>http://0.0.0.0:8000</code> . If it says <code>127.0.0.1</code> , it was started the wrong way.
4	Power the ESP32 (USB). Optional: open Arduino IDE Serial Monitor at 115200 baud.	Wi-Fi connected. IP: 10.144.56.x, then HTTP response code: 202 every 15 s.
5	Watch the server terminal.	10.144.56.21:... "POST /sensor-data HTTP/1.1" 202 Accepted every 15 s, each line now stamped with the wall-clock time it happened.
6	Open the dashboard on the PC: <code>http://127.0.0.1:8000/dashboard</code> .	Sensor cards fill in within 15 s; <i>Field monitor</i> shows the device online; the new Server uptime chip next to it confirms the backend process itself is alive.
7	Open the dashboard on the phone: <code>http://10.144.56.125:8000/dashboard</code> (type the <code>http://</code> part).	Same page as the PC.
8	Click Enter access key and type the value of <code>EXPECTED_DEVICE_KEY</code> (currently <code>demo-key-123</code>). Do it once per device.	Demo buttons and SMS language chips appear. The key is remembered for 12 hours.
9	Optional SMS check: press any demo button, e.g. low moisture.	An SMS marked [TEST] arrives within a few seconds.
10	Optional real-data SMS check: once a real device shows an active alert, press Test SMS with real data in the same panel.	An SMS arrives with today's genuine sensor numbers, no [TEST] prefix, bypassing the cooldown (§12, §14).

Shutting down

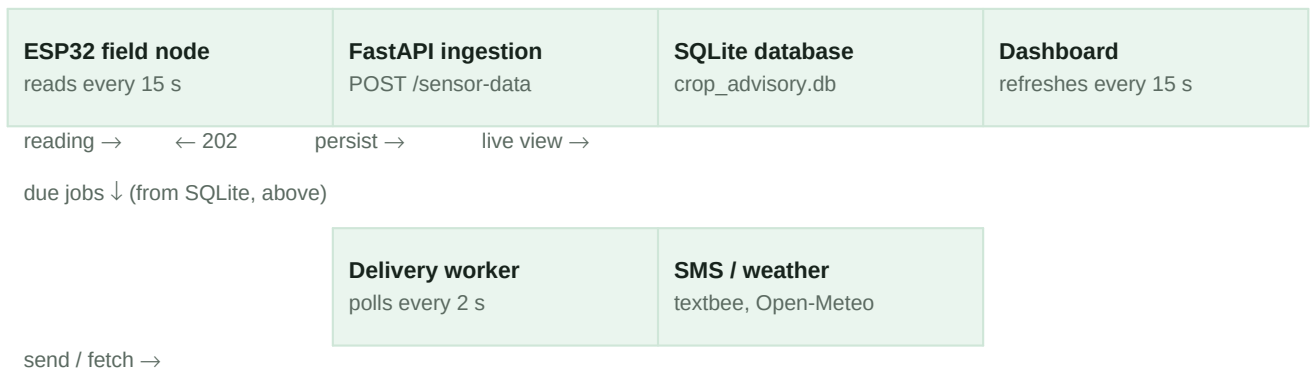
Press `Ctrl+C` in the server terminal, unplug the ESP32, turn off the hotspot. All readings, alerts and history are kept in `crop_advisory.db` and are still there next time.

2. Overview & Architecture

Crop Advisory is a field-monitoring prototype: an ESP32 node reads soil moisture, temperature/humidity, rain, and ambient light, and a FastAPI backend turns those readings into SMS advisories for a paddy farmer near Bhatkal,

Karnataka — using a deterministic, versioned rule engine, never an LLM, to decide what to say.

Sensor ingestion is decoupled from delivery, so a slow SMS provider or weather API can never delay or block the ESP32:



What changed from v2: the old backend kept all state in memory (wiped on every restart) and called weather/mandi/SMS *inside* the /sensor-data request — so a slow provider made the ESP32 wait, and a restart silently dropped active alerts. v3 introduced a persistent SQLite outbox, which v4 keeps: /sensor-data now does only cheap, synchronous work (authenticate, validate, dedupe, persist, evaluate rules) and returns **HTTP 202** in milliseconds; a background worker does the slow external work afterwards, with real retry/backoff on failure.

Important consequence: the SMS language, like the SMS text itself, is fixed at the moment an alert is created, not when it is sent (see §9.2).

NEW — this edition adds basic operational visibility on top of that pipeline: every log line now carries a wall-clock timestamp, every reading and SMS attempt is logged as it happens, and the dashboard shows how long the backend has been running plus a short recent-activity history — useful for confirming the system is alive without opening the database (§10, §17).

3. Hardware & Wiring

Components: ESP32-WROOM-32 DevKit, DHT22 (temperature/humidity), FC-28 soil moisture probe (via an LM393 comparator board), MH-RD rain plate (via an LM393 comparator board), LM393 LDR module (ambient light).

All three comparator boards: wire **A0** (the raw analog output), not D0. D0 is a fixed digital trigger set by the board's potentiometer — this project reads the raw analog value, so it gets an actual index rather than a single hard-coded on/off threshold. Every sensor's GND goes to the same ground rail as the ESP32; VCC goes to 3V3 on all four modules. The full wiring diagram is hardware/circuit_diagram.svg — open it in a browser and wire exactly as shown.

Sensor	Signal pin	ESP32 pin	Notes
DHT22	DATA	GPIO4	Add a 10 kΩ pull-up between DATA and 3V3 only if the module doesn't already have one
FC-28 (soil)	A0 on the LM393 board	GPIO34 (ADC1_6)	Probe plugs into the comparator board's 2-pin header
MH-RD (rain)	A0 on the LM393 board	GPIO35 (ADC1_7)	Plate plugs into its own comparator board
LM393 LDR	A0	GPIO32 (ADC1_4)	Ambient light — informational only, never gates an alert

Sanity check on readings

A soil index of exactly **0.0** means the raw value is at 4095 (your `SOIL_ADC_DRY`). That's correct in dry air, but in moist soil it usually means the wire comes from D0 instead of A0, or the probe is disconnected.

A light index stuck at exactly **100.0** or **0.0** can be real, but often means a floating or D0-wired pin. Re-run the calibration sketch (§4.3) to confirm the raw value moves when you cover and light the sensor.

4. Flashing the ESP32

4.1 Install board support

Arduino IDE → File → Preferences → Additional Board Manager URLs, add:

```
https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json
```

Then Tools → Board → Boards Manager → search "esp32" → install the Espressif Systems package. Select Tools → Board → **ESP32 Dev Module**, plug the board in via USB, and select the correct Port.

4.2 Install libraries

Sketch → Include Library → Manage Libraries: **DHT sensor library** (Adafruit), **Adafruit Unified Sensor** (auto-installs as a dependency), **ArduinoJson** (Benoit Blanchon).

4.3 Calibrate all three analog sensors

Open `hardware/analog_sensor_calibration.ino`, upload it, open Serial Monitor at 115200 baud. It prints Soil raw, Rain raw and LDR raw every second.

Sensor	How to measure	Setting in config.h
Soil (FC-28)	Probe in dry air	<code>SOIL_ADC_DRY</code>
	Probe fully submerged in water	<code>SOIL_ADC_WET</code>
Rain (MH-RD)	Dry plate, then a few drops flicked on; take the halfway point	<code>RAIN_ADC_THRESHOLD</code>
LDR	Fully covered / lights off	<code>LDR_ADC_DARK</code>
	Torch or phone flashlight directly on it	<code>LDR_ADC_BRIGHT</code>

Skipping this step makes every reading meaningless — each physical sensor unit (especially the comparator boards, which also have a sensitivity potentiometer) reads differently.

4.4 Create config.h

Copy `hardware/config.example.h` to `hardware/crop_advisory_node/config.h` (next to the `.ino`; it's gitignored, so your real Wi-Fi password and device key never get committed). A complete file looks like this:

```

#ifndef CONFIG_H
#define CONFIG_H

// ---- Wi-Fi (must be 2.4 GHz; the ESP32 cannot join 5 GHz) ----
#define WIFI_SSID      "Cookie"
#define WIFI_PASSWORD ""

// ---- Backend: the PC's IPv4 address from ipconfig (see section 5) ----
#define BACKEND_URL "http://10.144.56.125:8000/sensor-data"

// ---- Device identity: DEVICE_KEY must equal EXPECTED_DEVICE_KEY in .env ----
#define DEVICE_ID  "FIELD-001"
#define DEVICE_KEY "demo-key-123"

// ---- Pins ----
#define DHT_PIN  4
#define DHT_TYPE DHT22
#define SOIL_PIN 34
#define RAIN_PIN 35
#define LDR_PIN  32

// ---- Reading interval (start-to-start) ----
#define READING_INTERVAL_MS (15UL * 1000UL)

// ---- Calibration values from section 4.3 ----
#define SOIL_ADC_DRY      4095
#define SOIL_ADC_WET      1600
#define RAIN_ADC_THRESHOLD 2750
#define LDR_ADC_DARK      0
#define LDR_ADC_BRIGHT   4095

#endif

```

4.5 Upload the main firmware

Open `hardware/crop_advisory_node/crop_advisory_node.ino` (Arduino IDE shows `config.h` as a second tab automatically) and click Upload. Watch Serial Monitor at 115200 baud: it should connect to Wi-Fi, print all four readings, and show HTTP response code: 202 on each POST, every 15 s. **Any change to `config.h` only takes effect after you upload again.**

Firmware fix applied during setup (use the patched .ino)

The original sketch counted only HTTP 200 as success, but the v4 backend replies **202 Accepted**, so Serial Monitor printed "Data send failed" even when the reading was stored. The patched `crop_advisory_node.ino` accepts any 2xx code, uses shorter timeouts (5 s connect, 10 s response — the backend answers in milliseconds), and on failure prints the error code, the URL it tried and the ESP32's own IP.

Put the patched file in `hardware/crop_advisory_node/`, replacing the old one, and keep your existing `config.h`. You can tell which version is running: the patched one prints HTTP POST failed, error -1: ... with a number; the old one prints HTTP POST failed, error: ... without it.

Because of the backend's state-change and cooldown logic, an unchanged reading does not re-send an SMS every cycle — that's intentional (§14 turns this into a demo beat).

5. Network Setup — Getting the ESP32 to Reach the PC

This was the cause of "dashboard not receiving data and no SMS" during setup: the ESP32 was sending to 192.168.1.122, an address that didn't exist on the hotspot. No reading was stored, so no alert and no SMS were created. Three things must all be true:

- The PC and the ESP32 are on the same **Wi-Fi network**.
- `BACKEND_URL` in `config.h` contains the PC's **current** address on that network.
- The PC accepts incoming connections on port 8000 (server on 0.0.0.0 + firewall rule).

5.1 Reading `ipconfig`

Run `ipconfig` in Command Prompt. Windows lists many adapters; only one or two matter:

Adapter in <code>ipconfig</code>	Example address	Meaning
Wireless LAN adapter Wi-Fi	10.144.56.125	The PC on the Wi-Fi it has joined (the phone hotspot Cookie). Use this when the ESP32 is on the phone hotspot.
Wireless LAN adapter Local Area Connection* 2	192.168.137.1	Only present when Windows Mobile Hotspot is on. The PC's address on its own hotspot. Use this if the ESP32 joins the PC's hotspot instead.
Ethernet adapter Ethernet	Media disconnected	Unused. Ignore.
Unknown adapter Tailscale	169.254.x.x	VPN / self-assigned address. Ignore — never use a 169.254 address.
Local Area Connection* 1	Media disconnected	Ignore.

5.2 Which address to use

Press the ESP32's EN/reset button and read `Wi-Fi connected. IP: ...` in Serial Monitor. Match the first three numbers:

ESP32 IP starts with	It has joined	<code>BACKEND_URL</code>
10.144.56.	the phone hotspot (current setup)	<code>http://10.144.56.125:8000/sensor-data</code>
192.168.137.	the PC's Windows Mobile Hotspot	<code>http://192.168.137.1:8000/sensor-data</code>
anything else	a different network from the PC	Move one of them to the same network first

Phone hotspot vs. PC hotspot on demo day

A phone hotspot hands out addresses dynamically, so the PC's 10.144.56.125 can change after the hotspot or PC reconnects — meaning a `config.h` edit and re-upload. With Windows Mobile Hotspot (Settings → Network & internet → Mobile hotspot, band set to 2.4 GHz), the PC is always 192.168.137.1, so `config.h` never needs to change. The trade-off: the textbee SMS phone then needs its own mobile data to reach textbee.

5.3 Open the firewall (once, as Administrator)

Hotspots are usually treated as *Public* networks, which block incoming connections. Open Command Prompt as Administrator and run:

```
netsh advfirewall firewall add rule name="CropAdvisory8000" dir=in action=allow protocol=TCP localport=8000 profile=any
```

This one rule covers the ESP32, the phone and any other device on the network. It stays in place permanently.

5.4 Five-second network test

On the phone (connected to or running the same hotspot), open `http://<PC-IP>:8000/`. If the page loads, the network path is fine and any remaining ESP32 problem is in `config.h` or the uploaded sketch. If it doesn't load, the problem is the firewall, the server start command, or the IP address.

6. Backend Setup on Windows (VS Code, first time only)

- 1 Open the `crop-advisory-backend` folder itself in VS Code as the workspace root — not its parent folder. If the root is one level too high, `.vscode/settings.json` and workspace-relative paths point at the wrong place and Run/Debug uses the wrong Python.
- 2 Create and activate a virtual environment, then install dependencies (commands below).
- 3 Copy `.env.example` → `.env` and fill in real values (§7, §8).
- 4 `Ctrl+Shift+P` → **Python: Select Interpreter** → pick `crop-advisory-backend\venv\Scripts\python.exe` explicitly.
- 5 Run the test suite — no network needed, nothing here can send a real SMS. Expect everything to pass except one test skipped on Windows (Linux/macOS timezone only).
- 6 Start the server with `start_server.bat`. On first run this creates `crop_advisory.db` next to `main.py` and starts the delivery worker automatically.

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
pytest tests/ -v
start_server.bat
```

Always start with `start_server.bat`

It runs `uvicorn main:app --host 0.0.0.0 --port 8000`. A plain `uvicorn main:app` or the VS Code Run button listens only on `127.0.0.1` — the dashboard works on the PC, but the ESP32 and phone get "connection refused".

Live credentials warning

Once `.env` has real textbee/Twilio credentials, any reading that produces a new alert — including one you post by hand while testing — sends a real SMS to `ADVISORY_TO_NUMBER` within a few seconds. Use `POST /sensor-data-preview` for free-form testing; it never stores or delivers anything.

7. Changing Settings — Where Everything Lives

Settings live in exactly three places. Knowing which one a setting is in tells you how to apply a change:

Where	What's there	To apply a change
config.h (ESP32)	Wi-Fi name/password, BACKEND_URL, DEVICE_ID, DEVICE_KEY, pins, reading interval, calibration values	Save, then Upload the sketch again in Arduino IDE
.env (backend)	SMS provider and credentials, recipient number, device key, location, sowing date, cooldowns, default SMS language, voice, worker and database settings	Save the file, then restart the server : Ctrl+C, then start_server.bat. Saving alone does nothing.
Dashboard (unlocked)	Temporary SMS language; demo buttons (including the new Test SMS with real data)	Takes effect immediately; the language choice is lost on the next server restart

.env editing rules

Edit the .env inside crop-advisory-backend (next to main.py) — not .env.example and not a copy elsewhere. Each setting must appear **only once**. If a name appears twice, the **last** line wins silently. No quotes and no spaces around =. Make sure the file is saved (no dot on the VS Code tab) before restarting. Changing the device key means changing **both** EXPECTED_DEVICE_KEY in .env and DEVICE_KEY in config.h, then restarting the server and re-uploading the sketch. A mismatch gives HTTP 401.

7.1 Common changes, step by step

I want to...	Change this	Then
Change the SMS language permanently	TRANSLATE_SMS_TO=hi (Hindi), kn (Kannada) or en in .env	Restart server (see §9.2)
Change it just for now	Dashboard → unlock → tap English / Hindi / Kannada chip	Immediate
Send SMS to a different phone	ADVISORY_TO_NUMBER=+91XXXXXXXXXX (with country code)	Restart server
Switch SMS provider	SMS_PROVIDER=textbee or twilio plus that provider's credentials	Restart server
Get repeat SMS more or less often	ALERT_COOLDOWN_MINUTES (sensor alerts), TIME_BASED_ALERT_COOLDOWN_MINUTES (fertilizer/harvest)	Restart server
Change the crop calendar	SOWING_DATE=YYYY-MM-DD – drives fertilizer and harvest alerts	Restart server (fails fast if malformed)
Change the location for weather/mandi	LOCATION_NAME / LOCATION_LAT / LOCATION_LON, MANDI_*	Restart server
Turn on mandi prices	DATA_GOV_API_KEY=<your key>	Restart server
Turn on voice calls	ENABLE_VOICE_CALL=true (needs Twilio; Gemini key optional)	Restart server
Change “No recent data” timeout	STALE_AFTER_MINUTES	Restart server
Read sensors more or less often	READING_INTERVAL_MS in config.h	Re-upload sketch

I want to...	Change this	Then
Use a different Wi-Fi / PC address	WIFI_SSID, WIFI_PASSWORD, BACKEND_URL in config.h	Re-upload sketch
Rename the field node	DEVICE_ID in config.h (appears as a new device on the dashboard)	Re-upload sketch
Recalibrate a sensor	Values from §4.3 into config.h	Re-upload sketch
Start completely fresh (delete all history)	Stop server, delete crop_advisory.db and any crop_advisory.db-wal / -shm files next to it	Start server; a new empty database is created

8. Environment Variables Reference (.env)

Variable	Default	Purpose
SMS_PROVIDER	twilio	twilio or textbee — switch with zero code changes (current: textbee)
TEXTBEE_API_KEY / TEXTBEE_DEVICE_ID	—	textbee credentials. The Android phone with the textbee app sends the SMS from its own SIM, so it must be on, online and have SMS balance
TWILIO_ACCOUNT_SID / TWILIO_AUTH_TOKEN / TWILIO_FROM_NUMBER	—	Twilio credentials (if SMS_PROVIDER=twilio)
ADVISORY_TO_NUMBER	falls back to TWILIO_TO_NUMBER	The phone number every alert SMS/call goes to
TWILIO_VOICE / TWILIO_VOICE_LANGUAGE	Polly.Aditi / en-IN	Voice for phone calls (only if ENABLE_VOICE_CALL=true). For Kannada use a Google kn-IN voice, not Polly.Aditi
GEMINI_API_KEY	—	Voice-message rendering only; SMS never uses an LLM
DATA_GOV_API_KEY	—	Mandi price lookups; blank = "unavailable", never a fake price
EXPECTED_DEVICE_KEY	—	Shared secret the ESP32 sends as X-Device-Key; must match DEVICE_KEY in config.h. Also the dashboard access key
LOCATION_NAME / _LAT / _LON	Bhatkal, 13.9853, 74.5550	Weather and mandi lookups
MANDI_STATE / _DISTRICT / _COMMODITY	Karnataka / Uttara Kannada / Paddy	Mandi price filter
SOWING_DATE	2026-06-15	YYYY-MM-DD; drives fertilizer/harvest timing. Startup fails fast if malformed
ALERT_COOLDOWN_MINUTES	5	How often a live sensor alert (moisture, sensor fault) repeats while it stays true
TIME_BASED_ALERT_COOLDOWN_M INUTES	1440 (24 h)	How often a calendar alert (fertilizer, harvest) repeats
STALE_AFTER_MINUTES	2	Dashboard shows "No recent data" after this long with no reading
PUBLIC_MODE	false	true on a server reachable from the internet — refuses a weak device key (under 16 chars), hides /docs, gates the preview endpoint
ENABLE_VOICE_CALL	false	Turns on the fully built voice-call path; SMS-only otherwise
TRANSLATE_SMS_TO	blank (English)	Default SMS language: en, hi or kn. Unknown values fall back to English silently
DATABASE_URL	sqlite:///./crop_advisory.db	Swap to a Postgres URL for a hosted deployment — no code changes

Variable	Default	Purpose
SMS_RETRY_MAX_ATTEMPTS	5	Delivery attempts before a job is marked permanently failed
WORKER_POLL_INTERVAL_SECONDS	2	How often the background worker checks for due delivery jobs
ENABLE_DELIVERY_WORKER	true	false disables the worker — no SMS is ever sent (the test suite does this automatically)

NEW — no new environment variables were added for this edition's features — uptime tracking, log timestamps and the Activity log are all automatic and need no .env setting.

9. SMS Language

9.1 How the language is chosen

For every new alert the backend uses, in order: the dashboard chip choice (if one was made since the last restart), otherwise TRANSLATE_SMS_TO from .env, otherwise English. Only the SMS is translated; numbers are always filled in by code as ASCII digits, and the dashboard itself stays in English.

9.2 Changed .env to Hindi but still getting English?

Hindi generation itself is verified to work. Check these, in order:

Cause	Check / fix
Server not restarted	.env is only read at start-up. Ctrl+C, then start_server.bat. Make sure the file was saved first.
Duplicate line	The original .env already had TRANSLATE_SMS_TO=en near the bottom. If hi was added as a new line above it, the later en wins. Ctrl+F and keep exactly one line.
Old SMS	The language is fixed when the alert is created. An SMS queued before the restart stays English, and the next LOW_MOISTURE repeat only comes after the 5-minute cooldown. Press a demo button to get a fresh SMS immediately.
Dashboard chip set to English	A chip choice overrides .env until the next restart. The active chip is highlighted green; tap Hindi, or restart the server to clear it.
Wrong file edited	Only crop-advisory-backend\.env counts — not .env.example or a copy in another folder.

Quick diagnosis: restart, unlock the dashboard and look at the chips. Hindi highlighted = setting works (any English SMS was old). English highlighted = the server isn't reading hi (restart, duplicate line, or wrong file).

10. Viewing the Data

From	Open	Notes
The PC running the server	<code>http://127.0.0.1:8000/dashboard</code>	127.0.0.1 always means "this device", so it only works on the PC itself
Phone (the hotspot)	<code>http://10.144.56.125:8000/dashboard</code>	Type <code>http://</code> explicitly — some browsers try <code>https</code> and fail. Bookmark it or add it to the home screen
Any other laptop/phone on the same Wi-Fi	<code>http://<PC-IP>:8000/dashboard</code>	Same firewall rule covers it
API docs / manual testing	<code>http://<PC-IP>:8000/docs</code>	Swagger UI (disabled when <code>PUBLIC_MODE=true</code>)
Scenario list	<code>http://<PC-IP>:8000/demo/scenarios</code>	Every demo scenario as JSON
Liveness check	<code>http://<PC-IP>:8000/</code>	Quick "is the server reachable?" test

What the dashboard shows

- **Field monitor badge** (top bar): online, waiting for data, or "No recent data" after `STALE_AFTER_MINUTES` without a reading.
- **Server uptime badge** (top bar, next to Field monitor): how long the backend *process itself* has been running — shown as both a duration and the wall-clock time it started, e.g. Up 2h 14m 32s · since 09:18:42. This confirms the backend is alive independently of whether any device has reported yet.
- **Sensor cards**: soil moisture index, temperature, humidity, rain now, ambient light (informational only).
- **Active alerts** with their alert codes (e.g. `LOW_MOISTURE`).
- **Last advisory sent**: SMS status (sent / retrying / failed) and voice-call status (disabled in SMS-only mode). The SMS text itself sits in a collapsed disclosure under the banner — click "The SMS text" to expand it, so a long message doesn't push the rest of the page down by default.
- **Activity log** (bottom of a device's page, click to expand): the last 10 real readings and last 10 SMS attempts for that device, newest first — a fast way to confirm what's actually been happening without opening `crop_advisory.db` directly.
- Footer: rule version and device ID. The page refreshes itself every 15 s; the Refresh button forces it now.
- When unlocked: the **Introduce a problem** demo buttons, the new **Test SMS with real data** button (only shown for a real, non-demo device), and the SMS language chips. Lock forgets the key on that browser.

The access key is `EXPECTED_DEVICE_KEY` from `.env` (not the device name). Each browser needs it once; it's remembered for 12 hours by a cookie, not in the URL.

11. Codebase Tour

File / folder	What it does
<code>main.py</code>	FastAPI app: routes, auth, the fast <code>/sensor-data</code> ingestion path, demo endpoints (including <code>/demo/test-real-sms</code>), and the wall-clock log format set up at start-up
<code>models.py</code>	SensorData payload validation: soil/humidity/light 0–100, temperature –10 to 60 °C, required <code>device_id</code> and sequence

File / folder	What it does
<code>rule_engine.py</code>	The "brain": hysteresis, sensor-fault detection, versioned rules, rain-vs-irrigation. Pure, no I/O
<code>agronomy/</code>	Versioned crop thresholds (<code>paddy_profile.py</code>) + <code>AGRONOMY_SOURCES.md</code> documenting where every number comes from
<code>message_planner.py</code>	Builds the SMS (deterministic template) and voice message (LLM-rendered, alert codes only — never raw numbers)
<code>sms_i18n.py</code>	English/Hindi/Kannada SMS templates; numbers are always filled in by code
<code>llm.py</code>	Gemini calls for voice rendering only; deterministic fallback on any failure
<code>telephony.py</code>	textbee/Twilio SMS + Twilio voice; classifies failures as retryable (timeout, 429, 5xx) vs. permanent (bad number)
<code>weather.py</code>	Open-Meteo forecast + observed rainfall — "unavailable" is always distinct from "no rain"
<code>mandi.py</code>	<code>data.gov.in</code> mandi price lookup, picks the freshest record deterministically
<code>dashboard.py</code>	The live-status HTML page (no JavaScript, no build step): demo control panel, the Server uptime badge, and the collapsible Activity log
<code>demo_scenarios.py</code>	The decision table: one named, reproducible scenario per alert code
<code>config.py</code>	Loads every setting from <code>.env</code> — zero hard-coded credentials
<code>database.py</code> , <code>db_models.py</code>	SQLAlchemy engine/session; tables Device, Reading, DeviceAlertState, AlertEvent, DeliveryJob, DeliveryAttempt, SystemEvent
<code>repositories/</code>	One module per table — all raw DB queries, nothing else, including the recent-readings and recent-SMS-attempts lookups the Activity log uses
<code>services/delivery_queue.py</code>	The outbox: creates delivery jobs, sends them, retries with backoff, and logs the outcome of every attempt (sent or failed) as it happens
<code>services/worker.py</code>	Background thread that polls for due delivery jobs
<code>state_manager.py</code>	DB-backed read layer for the dashboard: device snapshots, SMS-language setting, cooldown lookups, server start time/uptime, and recent-activity history
<code>http_errors.py</code>	Safe one-line error descriptions that never leak an API key into logs
<code>start_server.bat</code>	Starts uvicorn on 0.0.0.0:8000 using the project's venv
<code>hardware/</code>	ESP32 firmware (<code>crop_advisory_node/</code>), calibration sketch, wiring/flow/block diagrams, README
<code>tests/</code>	280+ tests across rules, API, dashboard, integrations, delivery queue and pinned regressions

12. API Endpoint Reference

Endpoint	Auth	Purpose
GET /	none	Liveness check
POST /sensor-data	X-Device-Key	Real ESP32 ingestion. Returns 202 in milliseconds with accepted, reading_id, duplicate, days_since_sowing, current_alert_codes, alerts_created, delivery_jobs_created. Delivery happens afterwards; the reading is now also logged to the console as it's ingested.
POST /sensor-data-preview	none locally; key if PUBLIC_MODE	Evaluates rules and returns the would-be SMS text; never stores or delivers
POST /demo/trigger	X-Device-Key	Fixed low-moisture scenario on the DEMO device; delivers synchronously
GET /demo/scenarios	none	Lists every named scenario
POST /demo/scenario/{name}	X-Device-Key	Fires one scenario; optional ?lang=en hi kn. Names: sensor_fault_dht, sensor_fault_soil, low_moisture, excess_moisture, fertilizer_basal, fertilizer_tillering, fertilizer_panicle, harvest_approaching, harvest_check_due (rain_warning returns 400 by design)
POST /demo/test-real-sms	X-Device-Key	NEW. ?device=<id> — force-sends a real SMS built from that REAL device's most recent ACTUAL reading, bypassing the cooldown. Only sends if that reading's real values currently imply an alert (never fabricates one); replies with a “no active alert” notice otherwise. Unlike the /demo/scenario/* buttons, the message carries no [TEST] prefix, since it's built from genuine data.
POST /demo/reset	X-Device-Key	Clears every simulated DEMO* device; real devices untouched
POST /settings/sms-language	X-Device-Key	?lang=en hi kn — runtime SMS language until restart
POST /dashboard/lock	none	Forgets the dashboard's access-key cookie
GET /dashboard	none to view; key to unlock	The live status page — now also shows server uptime and the Activity log
GET /docs	disabled if PUBLIC_MODE	Swagger UI — easiest way to try endpoints by hand

The dashboard sends a restrictive CSP (default-src 'none'), and no endpoint ever echoes a secret back.

13. Alert Codes Reference

Code	Severity	Triggered by
SENSOR_FAULT_DHT22	Fault	DHT22 returns no reading after 3 tries (never faked as a plausible number)
SENSOR_FAULT_SOIL	Fault	Soil probe returns no reading
LOW_MOISTURE	Warning	Soil index drops below 28 (hysteresis exit at 35)

Code	Severity	Triggered by
EXCESS_MOISTURE	Wet	Soil index rises above 75 (exit at 65); SMS says whether it's confirmed rain, likely irrigation, or unknown
FERTILIZER_DUE_BASAL	Info	Day 0 (sowing)
FERTILIZER_DUE_TILLERING	Info	Days 18–25
FERTILIZER_DUE_PANICLE	Info	Days 40–50
HARVEST_APPROACHING	Info	Within 10 days of estimated maturity (day 115)
HARVEST_CHECK_DUE	Info	At/past estimated maturity — a prompt to check, never a harvest command
RAIN_WARNING	Warning	Real forecast confirms rain AND harvest is near. Not force-triggerable — faking weather would undermine grounding it in real data
WEATHER_UNAVAILABLE	Info	The forecast was fetched (because something else needed telling) and failed — distinct from "never checked"

All force-triggerable codes have a one-click button in the dashboard's demo panel (§15); RAIN_WARNING is the one exception, by design. The new **Test SMS with real data** button (§12, §14) works differently: it doesn't target one specific code — it sends whichever of these are currently true for a real device's real reading.

14. Running the Live Demo — Real Hardware

Before the room fills up: complete the Quick Start (§1) — hotspot on, IP checked, server on 0.0.0.0, firewall rule in place, ESP32 posting 202s, dashboard open on a second screen or the phone.

Timing note: delivery is asynchronous — the SMS lands roughly `WORKER_POLL_INTERVAL_SECONDS` (2 s) after the dashboard updates. That gap is worth narrating: it's the outbox pattern working, not lag.

- 1 **Action** — dip the FC-28 probe in water (or pull it into dry air) → dashboard updates within ~15 s → SMS a couple of seconds later. Say: “That decision came from a deterministic, versioned rule — `rule_version` is in every response — not an LLM.”
- 2 **No repeat spam** — leave it in the same state through the next cycle → dashboard refreshes, but no second SMS until the 5-minute cooldown ends. Say: “A field that stays dry doesn't text every 15 seconds — that was the core bug this system was built to fix.”
- 3 **Honest failure handling** — unplug the DHT22 → sensor-fault alert, never a fabricated temperature/humidity.
- 4 **Rain trigger** — flick a few drops on the MH-RD plate → “Raining now” flips to yes on the next reading.
- 5 **Resilience beat** — Ctrl+C the server, run `start_server.bat` again. Say: “That restarted the whole backend — the active alert and its history are still there.” (In v2 this wiped all state.) Note: a dashboard language-chip choice resets on restart.
- 6 **LDR** — an example of a sensor deliberately scoped as informational-only — a good answer to “why doesn't light level trigger anything?”
- 7 **Language** — tap Hindi or Kannada on the dashboard, then trigger an alert: the SMS arrives in that language with identical numbers.
- 8 **NEW — Real-data SMS check** — once step 1 has left a real alert active, unlock the panel and press **Test SMS with real data**: an SMS goes out immediately using today's genuine sensor numbers, bypassing the 5-minute cooldown, with no [TEST] prefix. Say: “That's the exact same message pipeline a farmer would get — I just forced it to fire on demand, using this field's real reading.”

If a judge asks why voice calling is off: “It's fully built and tested — see `ENABLE_VOICE_CALL` — scoped off today to keep live external dependencies manageable on stage, not because it's unbuilt.”

15. Running the Live Demo — No Hardware

Every alert the system can raise has a one-click, reproducible trigger — usable as the main demo path with no hardware at all, or as a fallback if a sensor misbehaves on stage.

- 1 Start the server (§6) and open the dashboard (§10).
- 2 Click **Enter access key** and type `EXPECTED_DEVICE_KEY` from `.env` (not the device name).
- 3 The **Introduce a problem** panel shows one button per force-triggerable alert. Each press makes up a reading with that exact problem and sends it through the real pipeline (same rule engine and delivery path as a real reading); sends a real SMS marked **[TEST]**; opens a simulated page (`DEMO-<NAME>`) with an amber “Simulated problem” banner; and fires again every press — no cooldown.
- 4 **Back to real values** clears every simulated device and returns to the real field readings; real device state is never touched.
- 5 The SMS language chips (English / Hindi / Kannada) switch every subsequent SMS, automatic and demo alike.

- 6 For scripted testing: GET /demo/scenarios lists the decision table, and /docs lets you fire POST /demo/scenario/{name} or POST /sensor-data-preview directly.

NEW — the **Test SMS with real data** button (§12, §14) is a real-hardware companion, not part of this no-hardware path — it only appears for a real device that already has at least one stored reading, and it deliberately never fabricates a problem. With no hardware connected, use the buttons above instead.

Live SMS caution

Every demo button sends a real message — there is no dry-run toggle in the current build. With textbee, each one uses the phone's SIM, so budget SMS balance before a run-through.

16. Serial Monitor Messages Decoded

Message	Meaning	What to do
Connecting to Wi-Fi..... then Wi-Fi connection failed	Can't join the network	Check WIFI_SSID/PASSWORD (case-sensitive), hotspot on, 2.4 GHz band
Wi-Fi connected. IP: x.x.x.x	Joined the network	Compare the first three numbers with the PC's IP (§5.2)
HTTP response code: 202	Reading stored by the backend	Nothing — working
HTTP response code: 401	Device key mismatch	Make DEVICE_KEY and EXPECTED_DEVICE_KEY identical (whitespace counts)
HTTP response code: 422	Payload rejected (out-of-range value)	Check calibration; values must be 0–100 (temperature –10 to 60)
HTTP POST failed, error -1: connection refused	Catch-all for "couldn't connect at all" — wrong IP, server not running / on 127.0.0.1, or firewall	§5: check IP, start_server.bat, firewall; run the phone test (§5.4)
HTTP POST failed, error -11: read Timeout	Connected but no answer in time	Server frozen or overloaded — check the server terminal
DHT22 read failed, retrying...	One glitchy read (normal occasionally)	Nothing unless it becomes a fault
DHT22 failed 3 reads in a row	Sends SENSOR_FAULT_DHT22	Check DHT22 wiring on GPIO4 and 3V3/GND
Data send failed right after a 202	Old firmware (only accepted 200)	Upload the patched .ino (§4.5)

17. Troubleshooting

Symptom	Likely cause	Fix
Dashboard says "Waiting for the first sensor reading" and no SMS, but Serial Monitor shows readings	ESP32 can't reach the backend (this was the original problem)	\$5: correct BACKEND_URL for the PC's current IP, re-upload, server via start_server.bat, firewall rule
ESP32 readings stop arriving after working	Hotspot/router gave the PC a new IP	ipconfig, update BACKEND_URL, re-upload. Consider the PC hotspot (§5.2)
Dashboard works on the PC but not the phone	Wrong address, https, or firewall	Use http://<PC-IP>:8000/dashboard, type http:// yourself; check the firewall rule
401 Unauthorized on /sensor-data	X-Device-Key doesn't match EXPECTED_DEVICE_KEY	Compare exactly (whitespace counts) against .env and config.h
Readings arrive (202) but no SMS	textbee phone offline/no balance, worker disabled, or cooldown	Check the textbee app and SIM; ENABLE_DELIVERY_WORKER=true; the dashboard's "Last advisory" status and Activity log; look for the timestamped "SMS sent" / "SMS failed" line in the server terminal
No SMS on a repeated reading	Expected — state-change/cooldown logic	Cross a threshold, wait for the cooldown, use a demo button, or press Test SMS with real data to force one immediately (§12)
SMS in English after setting Hindi	See §9.2	Restart, remove duplicate line, check chips
Soil stuck at 0 or light stuck at 100	Pinned ADC value — D0 wired instead of A0, or disconnected	Re-check wiring; re-run calibration (§4.3)
"Import could not be resolved" (Pylance)	VS Code's interpreter isn't the project venv	Python: Select Interpreter → crop-advisory-backend\venv\Scripts\python.exe
ModuleNotFoundError: No module named 'dotenv'	Run/Debug using a cached wrong interpreter	Re-run Python: Select Interpreter explicitly; better, use start_server.bat
Gemini call fails silently	Expected — falls back to a deterministic voice template	Check logs; SMS unaffected
Mandi price shows unavailable	DATA_GOV_API_KEY unset (current setup), or no record today	Expected; never fabricated
sqlite3.OperationalError: database is locked	Two server processes on the same DB	Close extra terminals; run only one server
An alert doesn't seem to deliver	Worker disabled, or genuinely mid-retry	ENABLE_DELIVERY_WORKER=true; retries up to SMS_RETRY_MAX_ATTEMPTS with backoff — never silently dropped
A test or stray POST sent a real SMS	Live credentials + a new alert	Use /sensor-data-preview for exploratory testing

Symptom	Likely cause	Fix
Console logs are hard to follow / no timestamps	Fixed in this edition — every log line now starts with YYYY-MM-DD HH:MM:SS in local (Asia/Kolkata) time, and every reading and SMS attempt is logged as it happens	Nothing to do — just read the terminal
Not sure if a send used real or fake data	Demo buttons vs. Test SMS with real data look different by design	A demo/fake send shows “Simulated problem ... Made-up readings” and [TEST] in the SMS body; a real send (including Test SMS with real data) shows “Device <id> · reading taken HH:MM:SS” with no such disclaimer and no [TEST] prefix

18. Known Limitations & Roadmap

Stated honestly, once, not apologized for repeatedly on stage:

- Agronomic thresholds are illustrative, adapted from general ICAR Kharif guidance — not validated for the exact paddy variety or local Bhatkal KVK recommendations (agronomy/AGRONOMY_SOURCES.md documents every number's source).
- No SMS dry-run mode — every real alert and demo button sends an actual message; the new Test SMS with real data button is not a dry run either, it also sends for real.
- No full Demo Control Center or offline-device monitor yet, though the new Activity log (§10) now gives basic recent-history visibility per device. Today's alert behavior is unchanged: new alert → immediate SMS, repeats every ALERT_COOLDOWN_MINUTES while true, silence otherwise.
- "SMS sent" in the logs/dashboard means the provider (Twilio/textbee) *accepted* the request — not that the handset received it. This matters most for textbee, where actual delivery depends on a separate Android phone being online, signed in and having SMS balance; there is no delivery-status callback wired up yet to confirm the final hop.
- No HMAC/replay-resistant device signing — authentication is a shared secret; per-device secrets are scaffolded in the DB but not provisioned through any UI.
- No database migrations (Alembic) — schema changes mean a fresh crop_advisory.db.
- FC-28 corrosion drift — exposed prongs corrode; recalibrate over weeks or use a coated/capacitive probe.
- No rate limiting on any endpoint, and the dashboard SMS-language choice resets to the .env default on every restart.
- The ESP32 needs the PC's LAN address in its firmware; a changing hotspot IP means re-flashing (a PC hotspot or a hosted HTTPS backend removes this).
- Solar/battery power not attempted — the field node needs mains/USB power.

None of these block a convincing demo (§14, §15) — they're the honest boundary of what v4 covers versus what's still ahead.